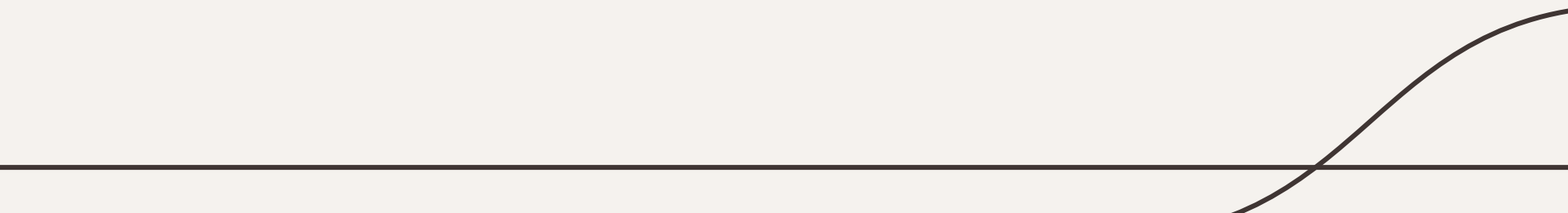


SSP Final Lab

Group 4



Outline

- S3prl
- Model selection
- Data pre-processing
 - Denoising
 - Direct subtraction
 - Sgmse
 - Noisereduce
 - SpeechBrain: MetricGAN+
- Finetune
- Output post-processing
- Future work



S3PRL

SPEECH TOOLKIT

Only testing

```
test-clean loss: 4.241379737854004
test-clean uer: 55.13500414752127
test-clean wer: 86.80914936039608
```

```
1-1-1929 SOMEWHERE OUT THERE IS A GIANT MASS OF SILVER FISH ALL SWIMMING IN THIS DIRECTION THEY DONT KNOW WHY THEYRE DOING IT WE DONT KNOW WHY TH
1-1-1946 DO YOU REMEMBER THE FIRST TIME THAT WE CAME TO SEE THE GRUNION IT WAS ABOUT FOUR YEARS AGO RIGHT AFTER WE GOT MARRIED WE THOUGHT I WAS P
1-1-0892 THEY ARE SWIMMING RIGHT NOW AS IF ITS THE ONLY THING THAT MATTERS ITS ITS SINGLE MINDEDNESS WITH NO MIND URGENCY THATS PURE URGE ITS THE
1-1-1997 DO YOU THINK THAT I THOUGHT THERE WOULD BE TIMES WHEN YOU WOULD LOOK AT ME LIKE I WAS USED KLEENEX OR I WOULD LOOK AT YOU AND THINK GOOD
1-1-1553 HI I NEED I NEED SOME HELP IVE BEEN TRANSFERRED TO LIKE EIGHT DIFFERENT DEPARTMENTS AND IVE TOLD MY PROBLEM TO EVERY SINGLE DEPARTMENT A
1-1-0326 OF COURSE YOURE WORRIED WERE ALL WORRIED IM WORRIED FOR YOU EVERYONES ALWAYS WORRIED YOU KNOW I DONT MAYBE I HAVE A JOB AND YEAH ITS A G
1-1-0908 WE HAD A BOTTLE OF CHAMPAGNE BUT NO GLASSES AND YOU ASK ME TO DANCE SO WE TOOK OFF OUR SHOES AND WHILE WE WERE DANCING YOU SAID THE MOST
1-1-1986 YOU WERE FINE ON THE PHONE CALL WITH MARGE EATING DINNER WAS FINE TALKING UH WASHING THE DISHES NO PROBLEM ALL RIGHT SO IM TAKING A SHOW
1-1-1899 WHAT TIME IS IT THEYRE SUPPOSED TO RUN AROUND MIDNIGHT OH THIS IS GREAT ISNT IT LOOK AT THE NIGHT WEVE GOT HERE IT COULDNT BE BETTER ACT
1-1-0730 YES I LIKE IT AN HOUR A DAY IF I HAD TO GRUB FOR MONEY ALL DAY LONG I LIKE TO COME HOME TO SOMETHING BEAUTIFUL I WANT A FAMILY I WANT I
```

```
1-1-1929 L
1-1-1946 HERE ARE R THE HARST TIME THET THEKIME SE THE GU FROUGHT FOR YOURS ADOALRY ACTUR WAD DII DE THOUGHT I WAS PRI THE AL S
1-1-0892 THE SIMIG RIGHT NOW HIT THIFIL AS IT ID THE ONLY THING THE NAT ITIYS IT SINGLE MINDED ISNOLE MNGE ERGENS YET SPR PERGE
1-1-1997 YU THINGK TETIN TIGET HO A TRED DE ITIN DO WIT LOOK ELY LIKE A WO NLE NECORKFR I WOLDLOCH DOWIT TIG COOD GOD WITH AN B
1-1-1553 H I KNEED I NEED SHOM HOLP VE BEEN TRANSFERD TILLIKE EIGHTE DIFFERENT APARTMENTS AND I'VE TOLD MY PROLEM TO EVERY SING
1-1-0326 IF COURSHOU WARE S ALL WOR ID IM WONE HEBORE HINN HE MINOD ALWEY SHORYINAI ONTN THE I AVB AGO AND APGHO JO WET A ER AN
1-1-0908 WE HAD ABOL AS SHAMHANE BUT NO BLASSES AND YOU AST TO THE DAN SO WHE TOKG OF OUR SHES AND WHILE WEY ERE DANTING HE SAI
1-1-1986 YO'LD BYINO HONHAL WITH MACH HEDING DINER WAS BING TALKING WHATHING HE DISHES NO PROUP OR ICHIAL I'ME TAKING A SHOWER
1-1-1899 WHATIES IT THER'S WS TO RUN AROUN BID NIGHT BU TAT THIS IS GREAT ISN IT LIKE TAE NIGHE WE'VH GOT HERE CUNT BE BETTER I
1-1-0730 Y YES I LIKE IT AN HOUR A DAYY IF ID TO GRUBD FRM MONY AL DAY LONG I LIKE TO COME HOME OSOMETHING FVEAUTIFULS I I WONT
```

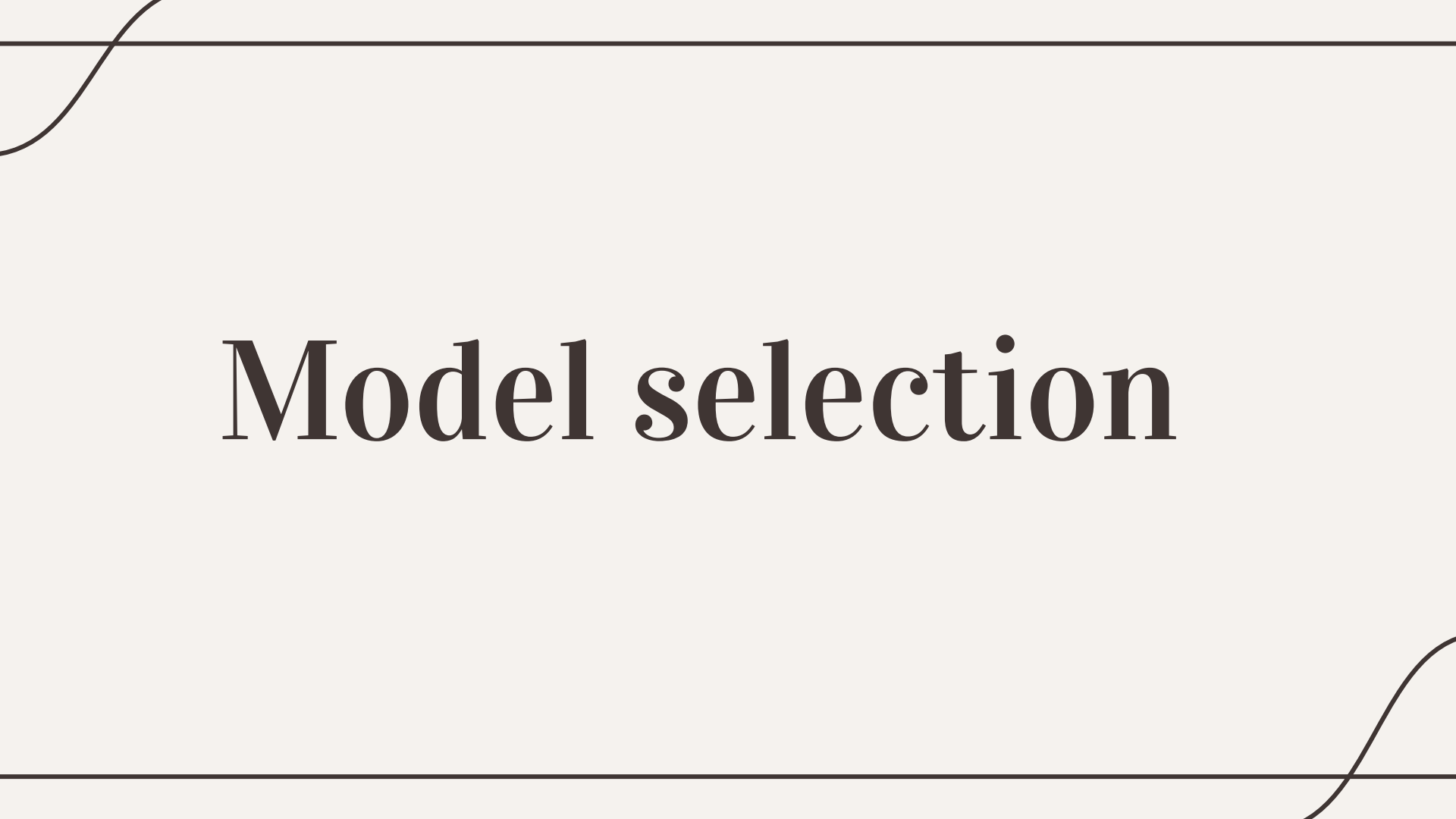
Training + Testing

1-1-1929 SOMEWHERE OUT THERE IS A GIANT MASS OF SILVER FISH ALL SWIMMING IN THIS DIRECTION THEY DONT KNOW WHY THEYRE DOING IT WE DONT KNOW WHY TH
1-1-1946 DO YOU REMEMBER THE FIRST TIME THAT WE CAME TO SEE THE GRUNION IT WAS ABOUT FOUR YEARS AGO RIGHT AFTER WE GOT MARRIED WE THOUGHT I WAS P
1-1-0892 THEY ARE SWIMMING RIGHT NOW AS IF ITS THE ONLY THING THAT MATTERS ITS ITS SINGLE MINDEDNESS WITH NO MIND URGENCY THATS PURE URGE ITS THE
1-1-1997 DO YOU THINK THAT I THOUGHT THERE WOULD BE TIMES WHEN YOU WOULD LOOK AT ME LIKE I WAS USED KLEENEX OR I WOULD LOOK AT YOU AND THINK GOOD
1-1-1553 HI I NEED I NEED SOME HELP IVE BEEN TRANSFERRED TO LIKE EIGHT DIFFERENT DEPARTMENTS AND IVE TOLD MY PROBLEM TO EVERY SINGLE DEPARTMENT A
1-1-0326 OF COURSE YOURE WORRIED WERE ALL WORRIED IM WORRIED FOR YOU EVERYONES ALWAYS WORRIED YOU KNOW I DONT MAYBE I HAVE A JOB AND YEAH ITS A G
1-1-0908 WE HAD A BOTTLE OF CHAMPAGNE BUT NO GLASSES AND YOU ASK ME TO DANCE SO WE TOOK OFF OUR SHOES AND WHILE WE WERE DANCING YOU SAID THE MOST
1-1-1986 YOU WERE FINE ON THE PHONE CALL WITH MARGE EATING DINNER WAS FINE TALKING UH WASHING THE DISHES NO PROBLEM ALL RIGHT SO IM TAKING A SHOW
1-1-1899 WHAT TIME IS IT THEYRE SUPPOSED TO RUN AROUND MIDNIGHT OH THIS IS GREAT ISNT IT LOOK AT THE NIGHT WEVE GOT HERE IT COULDNT BE BETTER ACT
1-1-0730 YES I LIKE IT AN HOUR A DAY IF I HAD TO GRUB FOR MONEY ALL DAY LONG I LIKE TO COME HOME TO SOMETHING BEAUTIFUL I WANT A FAMILY I WANT I
1-1-0922 I MEAN IT IS JUST THIS I MEAN IT INCLUDES A LOT AND EVERYTHING AND YOU KNOW ITS THE SAND AND ITS THE FULL MOON AND I JUST I AM SORRY BUT
1-1-0608 RIGHT RIGHT AND SINCE YEAH SINCE IT WAS JUST A COMPUTER GLITCH THEN THATS THAT SHOULD BE CLEARED UP AS WELL SO IM GOING TO GO AHEAD AND
1-1-1640 NO I DONT WANT A LOOK YOUR FATHER THINKS HES COMING BACK CHRIS YOU MARRY THAT GIRL AND YOURE PRONOUNCING HIM DEAD NOW WHATS GOING TO HAP

1-1-1929
1-1-1946 R TH TH Y W M TH N M
1-1-0892 TH T TH R
1-1-1997 UIN L N W N L N N THW TH LTHN W TH O N
1-1-1553 R L NN LN NTN TH NMN THE WNN THER NNM R H TH H RN TH N W M N MNN HN THN N N WN
1-1-0326 W N H THSN N A N N W N NTI NHANN N THA N OU R
1-1-0908 W HA T N H R W TH N M H
1-1-1986 U W N NR S WN THN W HE NNN L TN N N N H TI
1-1-1899 T THS STH N M THS N N BL WN LT HA FRTHE TH TH
1-1-0730 LN H TH R L THI WN W WN N N M SN THNR B
1-1-0922 HN N HNN N R
1-1-0608 WN T THR S W HA T THA NNR N RNN TH NRTHYUS HN TH
1-1-1640 O N YU R N THA N N

```
wer = jiwer.wer(merged_targets, merged_predictions)
print(f"Calculated WER using jiwer: {wer}")
```

Calculated WER using jiwer: 0.9929654770705917

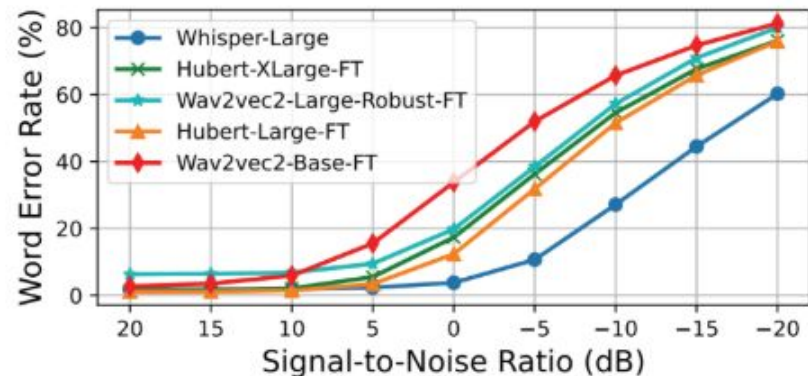


Model selection

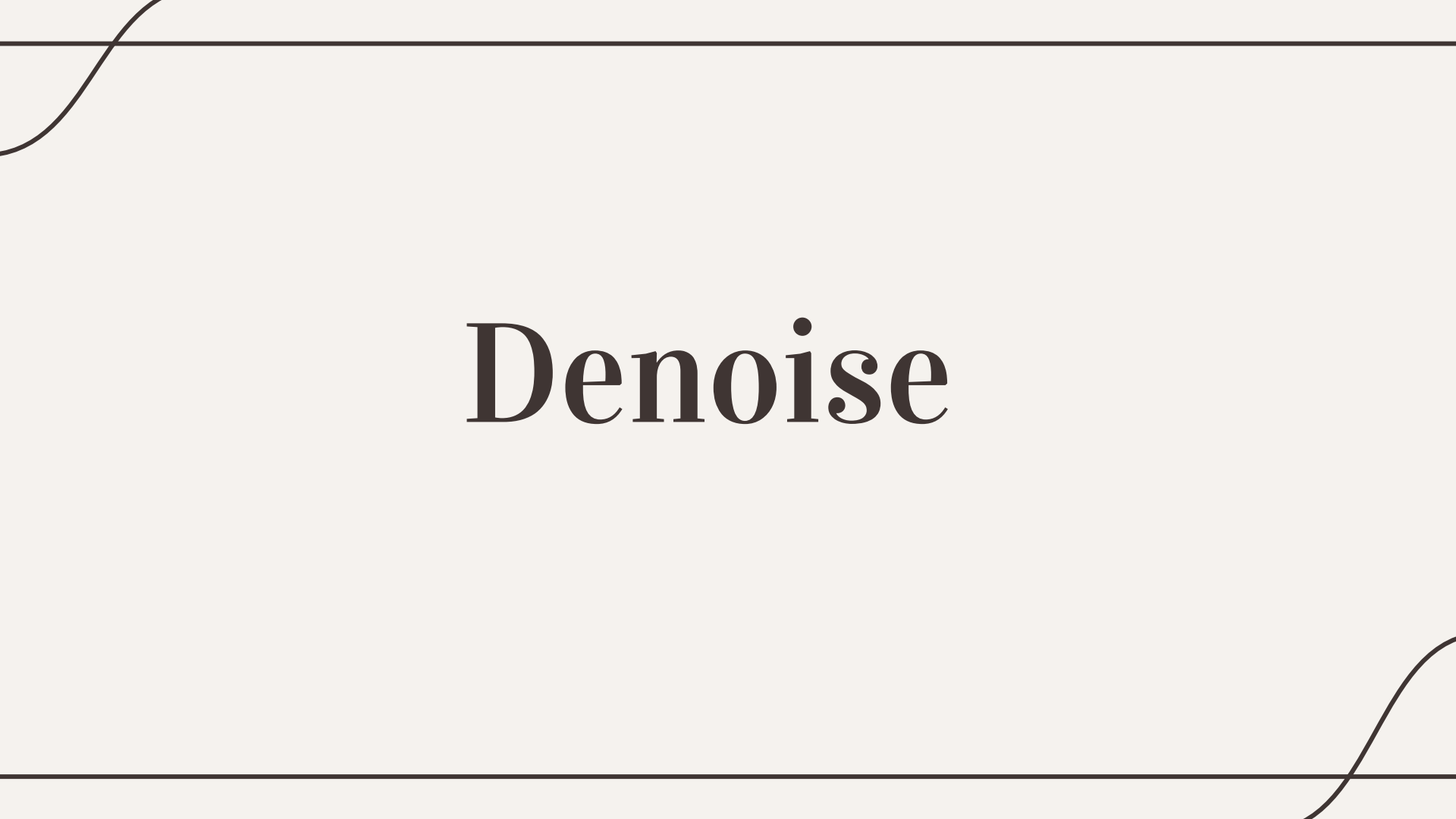
Model selection

	turbo	large-v3	Parakeet	Canary
final	0.247	0.24054	0.26166	0.297

model	Average WER	RTFx
nvidia/canary-v1b	6.5	235.34
nyrahealth/CrisperWhisper	6.67	84.05
nvidia/parakeet-tdt-1.1b	7.01	2390.61
nvidia/parakeet-rmt-1.1b	7.12	2053.15
nvidia/parakeet-ctc-1.1b	7.4	2728.52
openai/whisper-large-v3	7.44	145.51



Whisper-AT: Noise-Robust Automatic Speech Recognizers are Also Strong General Audio Event Taggers

The image features two thin, dark horizontal lines. The top line starts with a smooth, curved drop on the left side. The bottom line ends with a smooth, curved rise on the right side.

Denoise

Denoise

- Direct subtraction

	turbo	large-v3	Parakeet	Canary
final	0.247	0.24054	0.26166	0.297
direct denoise	0.298	0.30109	0.29661	0.3477

[dia0_utt1_noise-free-sound-0548](#)
[dia0_utt2_music-rfm-0137](#)
[dia1_utt0_noise-free-sound-0672](#)
[dia1_utt1_music-fma-wa-0092](#)
[dia1_utt2_noise-sound-bible-0072](#)
[dia1_utt3_music-jamendo-0144](#)
[dia1_utt4_noise-free-sound-0116](#)
[dia1_utt5_noise-free-sound-0580](#)
[dia1_utt6_noise-free-sound-0314](#)
[dia1_utt7_music-fma-wa-0035](#)
[dia2_utt0_music-hd-0039](#)
[dia2_utt1_noise-free-sound-0675](#)
[dia2_utt2_noise-free-sound-0673](#)
[dia2_utt4_noise-free-sound-0526](#)
[dia2_utt5_music-jamendo-0191](#)
[dia2_utt6_music-jamendo-0171](#)
[dia2_utt7_noise-free-sound-0305](#)
[dia2_utt8_noise-free-sound-0539](#)
[dia2_utt9_music-jamendo-0158](#)
[dia2_utt10_noise-free-sound-0628](#)
[dia3_utt0_music-jamendo-0069](#)

```

def s_n_p_align(s, ns):
    """調整語音與噪音的長度一致"""
    if len(s) > len(ns):
        new_ns = np.tile(ns, len(s) // len(ns) + 1)[:len(s)]
    else:
        new_ns = ns[:len(s)]
    return s, new_ns

# 載入音檔與噪音檔案，確保單聲道
y, sr = librosa.load(input_path, sr=16000)
noise, _ = librosa.load(noise_path, sr=16000)

y = ensure_mono(y)
noise = ensure_mono(noise)

# 調整語音與噪音的長度一致
y, noise = s_n_p_align(y, noise)

# 計算 STFT (短時傅立葉變換)
D_signal = librosa.stft(y)
D_noise = librosa.stft(noise)

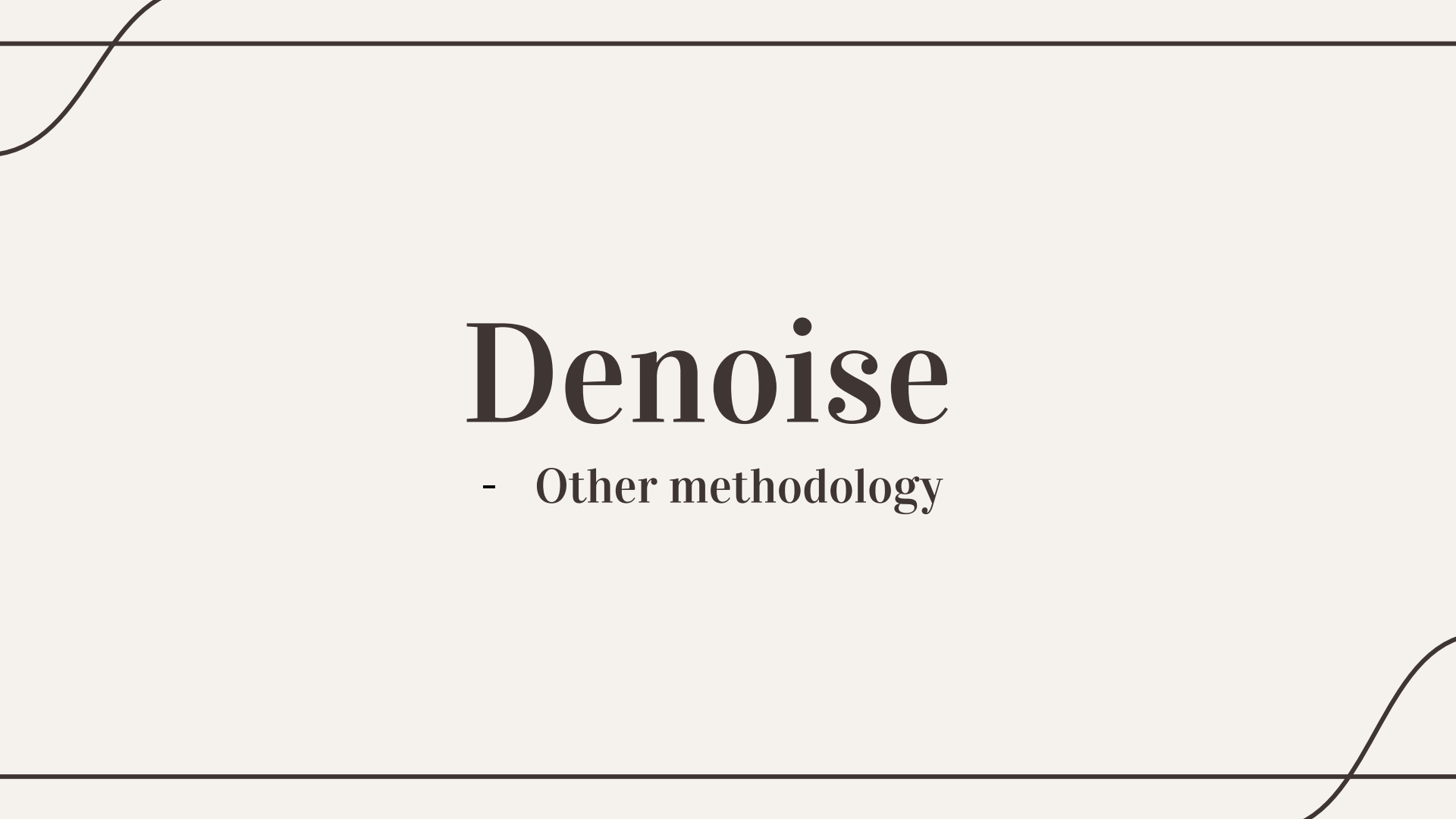
# 頻譜分析與減法
S_signal, phase_signal = librosa.magphase(D_signal)
S_noise, _ = librosa.magphase(D_noise)

# 頻譜減法，確保結果不會為負值
S_denoised = np.maximum(S_signal - S_noise, 0)
D_denoised = S_denoised * phase_signal

# 還原到時域
y_denoised = librosa.istft(D_denoised)

```

	Noise	Music
Original Final Dataset		
Denoise Final Dataset		



Denoise

- Other methodology

WER Performance

	turbo	large-v3	Parakeet	Canary
final_target_set	0.247	0.24054	0.26166	0.297
direct denoise	0.298	0.30109	0.29661	0.3477
Wiener denoise	0.315	0.305	—	—
Noisereduce	0.3078	0.2842	—	—
MetricGAN+ denoise	0.5262	0.4946	—	—

Denoised Sound

ID	2	3	4
Final_Target_Set			
After noisereduce			
After Wiener			
After MetricGAN+			
After spectral gating			
After sgmse			

The image features two thin, dark horizontal lines. The top line starts with a curved segment on the left side, and the bottom line ends with a curved segment on the right side.

Finetune

Finetune

- Training w/ Session_1

```
[ ] class SpeechDataset(torch.utils.data.Dataset):
    def __init__(self, audio_info_list, tokenizer, sample_rate) -> None:
        super().__init__()

        self.audio_info_list = audio_info_list
        self.sample_rate = sample_rate
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.audio_info_list)

    def __getitem__(self, id):
        audio_id, audio_path, text = self.audio_info_list[id]

        # audio
        audio = load_wave(audio_path, sample_rate=self.sample_rate)
        audio = whisper.pad_or_trim(audio.flatten())
        #mel = whisper.log_mel_spectrogram(audio)
        mel = whisper.log_mel_spectrogram(audio, vmodel.dims.n_mels).to(vmodel.device)

        text = [*self.tokenizer.sot_sequence_including_notimestamps] + self.tokenizer.encode(text)
        labels = text[1:] + [self.tokenizer.eot]

        return {
            "input_ids": mel,
            "labels": labels,
            "dec_input_ids": text
        }
```

```
[ ] import jiwer
wer_score = jiwer.wer(truth=refs, hypothesis=res)
print(f" {wer_score=} ")
```

```
wer_score=0.40569482716132355
```

```
setup(self, stage=None):

    if stage == 'fit' or stage is None:
        self.t_total = (
            (len(self.__train_dataset) // (self.cfg.batch_size))
            // self.cfg.gradient_accumulation_steps
            * float(self.cfg.num_train_epochs)
        )

    train_dataloader(self):
        dataset = SpeechDataset(self.__train_dataset, self.tokenizer, self.cfg.sample_rate)
        return torch.utils.data.DataLoader(dataset,
                                            batch_size=self.cfg.batch_size,
                                            drop_last=True, shuffle=True, #num_workers=self.cfg.num_
                                            collate_fn=WhisperDataCollatorWithPadding(), num_workers=0
                                            )

    val_dataloader(self):
        dataset = SpeechDataset(self.__eval_dataset, self.tokenizer, self.cfg.sample_rate)
        return torch.utils.data.DataLoader(dataset,
                                            batch_size=self.cfg.batch_size,
                                            #num_workers=self.cfg.num_worker,
                                            collate_fn=WhisperDataCollatorWithPadding(), num_workers=0
                                            )
```




Post-processing

Minor post-processing

- Transform digits to words(ex:100->one hundred)
- interjection : (ex:uhuhh/uh uhh -> uh uhh)
- Conclusion : No significant improvement for this part

	Turbo
Original transcript	0.24744
After this post-processing transcript	0.2464

Best result

	final_target_set
turbo	0.247
large-v3	0.24054
Parakeet	0.26166
Canary	0.297

```
✓ 0 秒
▼ Eval WER

[6] import jiver
import glob
# check align
ground_truths_root = 'Final_Target_Set/transcript/'
groundtruths_paths = sorted([path for path in glob.glob(ground_truths_root+'/*.txt')])
hypothesis_paths = sorted([path for path in glob.glob(decode_root+'/*.txt')])
for ele in groundtruths_paths:
    assert ele.split('/')[1].split('.')[0] == hypothesis_paths[groundtruths_paths.index(ele)]
# load decode result & GT
groundtruths = []
hypothesis = []
for path in groundtruths_paths:
    with open(path, 'r') as f:
        groundtruths.append(f.read().replace('\n', ''))
for path in hypothesis_paths:
    with open(path, 'r') as f:
        hypothesis.append(f.read()[1:].replace('\n', ''))

[7] ver_score = jiver.wer(truth=groundtruths, hypothesis=hypothesis)
print(f"wer_score={}")

wer_score=0.24054035889508704
```

Future Work

- Investigate finetune under noisy dataset
- Investigate S3PRL
- Post-processing and Analysis
- Denoise Pre-processing by sgmmse

Post-processing

- Normalized (jiwer)

```
from jiwer import wer, RemovePunctuation, RemoveMultipleSpaces, Strip, ToLowerCase, Compose
transform = Compose([
    RemovePunctuation(),
    RemoveMultipleSpaces(),
    Strip(),
    ToLowerCase()
])
```

⇒ Normalized Refs: ['hi i need an id', 'okay im sorry but']
Normalized Res: ['hi i need an id', 'okay im sorry but']
WER: 0.24060756771288394

正規化處理

```
normalized_refs = [transform(ref) for ref in groundtruths]
normalized_res = [transform(re) for re in hypothesis]
```

計算 WER

```
error_rate = jiwer.wer(normalized_refs, normalized_res)
```

Post-processing

- Expand Common English Contraction Transform (jiwer)

Contraction	transformed into
won't	_will not
can't	_can not
let's	_let us
n't	_not
're	_are
's	_is
'd	_would
'll	_will
't	_not
've	_have
'm	_am

Example

```
import jiwer

sentences = ["she'll make sure you can't make it", "let's party!"]

print(jiwer.ExpandCommonEnglishContractions()(sentences))
# prints: ["she will make sure you can not make it", "let us party!"]
```

Source code in `jiwer/transforms.py`

```
error_rate = jiwer.wer(normalizedl_refs, normalizedl_res)
print(f"WER: {error_rate}")
```



WER: 0.22905039924444062

Post-processing

```
def standardize_text(text):
    text = re.sub(r"(?!\\w)im(?:\\w)", "i am", text) # 'im' 替换为 'i am'
    text = re.sub(r"(?!\\w)theres(?:\\w)", "there is", text) # 'theres' 替换为 'there is'
    text = re.sub(r"(?!\\w)heis(?:\\w)", "he is", text) # 'heis' 替换为 'he is'
    text = re.sub(r"(?!\\w)shes(?:\\w)", "she is", text) # 'shes' 替换为 'she is'
    text = re.sub(r"(?!\\w)youre(?:\\w)", "you are", text) # 'youre' 替换为 'you are'
    text = re.sub(r"(?!\\w)ive(?:\\w)", "I have", text) # 'ive' 替换为 'I have'
    text = re.sub(r"(?!\\w)ill(?:\\w)", "I will", text) # 'ill' 替换为 'I will'
    text = re.sub(r"(?!\\w)wont(?:\\w)", "will not", text) # 'wont' 替换为 'will not'
    text = re.sub(r"(?!\\w)dont(?:\\w)", "do not", text) # 'dont' 替换为 'do not'
    text = re.sub(r"(?!\\w)cant(?:\\w)", "cannot", text) # 'cant' 替换为 'cannot'
    text = re.sub(r"(?!\\w)its(?:\\w)", "it is", text) # 'its' 替换为 'it is'

    # 其他常见缩写词替换
    text = re.sub(r"(?!\\w)gonna(?:\\w)", "going to", text) # 'gonna' 替换为 'going to'
    text = re.sub(r"(?!\\w)wanna(?:\\w)", "want to", text) # 'wanna' 替换为 'want to'
    text = re.sub(r"(?!\\w)kinda(?:\\w)", "kind of", text) # 'kinda' 替换为 'kind of'
    text = re.sub(r"(?!\\w)gotta(?:\\w)", "got to", text) # 'gotta' 替换为 'got to'
    text = re.sub(r"(?!\\w)lemme(?:\\w)", "let me", text) # 'lemme' 替换为 'let me'
    text = re.sub(r"(?!\\w)gimme(?:\\w)", "give me", text) # 'gimme' 替换为 'give me'
    text = re.sub(r"(?!\\w)outta(?:\\w)", "out of", text) # 'outta' 替换为 'out of'
    text = re.sub(r"(?!\\w)shoulda(?:\\w)", "should have", text) # 'shoulda' 替换为 'should have'
    text = re.sub(r"(?!\\w)woulda(?:\\w)", "would have", text) # 'woulda' 替换为 'would have'
    text = re.sub(r"(?!\\w)coulda(?:\\w)", "could have", text) # 'coulda' 替换为 'could have'
    text = re.sub(r"(?!\\w)ya(?:\\w)", "you", text) # 'ya' 替换为 'you'
    text = re.sub(r"(?!\\w)aint(?:\\w)", "is not", text) # 'aint' 替换为 'is not' 或根据上下文修改
    text = re.sub(r"(?!\\w)yall(?:\\w)", "you all", text) # 'yall' 替换为 'you all'
    text = re.sub(r"(?!\\w)howdya(?:\\w)", "how do you", text) # 'howdya' 替换为 'how do you'
    text = re.sub(r"(?!\\w)whatcha(?:\\w)", "what are you", text) # 'whatcha' 替换为 'what are you'
    text = re.sub(r"(?!\\w)whered(?:\\w)", "where did", text) # 'whered' 替换为 'where did'
```



Thanks



Appendix

Finetune

- Training w/ Session_1+MELD

```
[ ] class SpeechDataset(torch.utils.data.Dataset):
    def __init__(self, audio_info_list, tokenizer, sample_rate) -> None:
        super().__init__()

        self.audio_info_list = audio_info_list
        self.sample_rate = sample_rate
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.audio_info_list)

    def __getitem__(self, id):
        audio_id, audio_path, text = self.audio_info_list[id]

        # audio
        audio = load_wave(audio_path, sample_rate=self.sample_rate)
        audio = whisper.pad_or_trim(audio.flatten())
        #mel = whisper.log_mel_spectrogram(audio)
        mel = whisper.log_mel_spectrogram(audio, wmodel.dims.n_mels).to(wmodel.device)

        text = [*self.tokenizer.sot_sequence.including_notimestamps] + self.tokenizer.encode(text)
        labels = text[1:] + [self.tokenizer.eot]

        return {
            "input_ids": mel,
            "labels": labels,
            "dec_input_ids": text
        }
```

```
wer_score = jiwer.wer(truth=groundtruths, hypothesis=hypothesis)
print(f" {wer_score=} ")
```

```
wer_score=1.455899814055604
```

```
setup(self, stage=None):

    if stage == 'fit' or stage is None:
        self.t_total = (
            (len(self.__train_dataset) // (self.cfg.batch_size))
            // self.cfg.gradient_accumulation_steps
            * float(self.cfg.num_train_epochs)
        )

    train_dataloader(self):
        dataset = SpeechDataset(self.__train_dataset, self.tokenizer, self.cfg.sample_rate)
        return torch.utils.data.DataLoader(dataset,
                                            batch_size=self.cfg.batch_size,
                                            drop_last=True, shuffle=True, #num_workers=self.cfg.num_
                                            collate_fn=WhisperDataCollatorWhithPadding(), num_workers=0
                                            )

    val_dataloader(self):
        dataset = SpeechDataset(self.__eval_dataset, self.tokenizer, self.cfg.sample_rate)
        return torch.utils.data.DataLoader(dataset,
                                            batch_size=self.cfg.batch_size,
                                            #num_workers=self.cfg.num_worker,
                                            collate_fn=WhisperDataCollatorWhithPadding(), num_workers=0
                                            )
```

Post-processing

- Expand Common English Contraction Transform (jiwer)

```
def standardize_text(text):
    text = re.sub(r"(?<!\w)im(?!\w)", "i am", text) # 'im' 替换为 'i am'
    text = re.sub(r"(?<!\w)theres(?!\w)", "there is", text) # 'theres' 替换为 'there is'
    text = re.sub(r"(?<!\w)heis(?!\w)", "he is", text) # 'heis' 替换为 'he is'
    text = re.sub(r"(?<!\w)shes(?!\w)", "she is", text) # 'shes' 替换为 'she is'
    text = re.sub(r"(?<!\w)youre(?!\w)", "you are", text) # 'youre' 替换为 'you are'
    text = re.sub(r"(?<!\w)ive(?!\w)", "I have", text) # 'ive' 替换为 'I have'
    text = re.sub(r"(?<!\w)ill(?!\w)", "I will", text) # 'ill' 替换为 'I will'
    text = re.sub(r"(?<!\w)wont(?!\w)", "will not", text) # 'wont' 替换为 'will not'
    text = re.sub(r"(?<!\w)dont(?!\w)", "do not", text) # 'dont' 替换为 'do not'
    text = re.sub(r"(?<!\w)cant(?!\w)", "cannot", text) # 'cant' 替换为 'cannot'
    text = re.sub(r"(?<!\w)its(?!\w)", "it is", text) # 'its' 替换为 'it is'
```

Post-processing

- Normalized (jiwer)

```
from jiwer import wer, RemovePunctuation, RemoveMultipleSpaces, Strip, ToLowerCase, Compose
transform = Compose([
    RemovePunctuation(),
    RemoveMultipleSpaces(),
    Strip(),
    ToLowerCase()
])
```

⇒ Normalized Refs: ['hi i need an id', 'okay im sorry but']
Normalized Res: ['hi i need an id', 'okay im sorry but']
WER: 0.24060756771288394

正規化處理

```
normalized_refs = [transform(ref) for ref in groundtruths]
normalized_res = [transform(re) for re in hypothesis]
```

計算 WER

```
error_rate = jiwer.wer(normalized_refs, normalized_res)
```

Post-processing

- Expand Common English Contraction Transform (jiwer)

Contraction	transformed into
won't	_will not
can't	_can not
let's	_let us
n't	_not
're	_are
's	_is
'd	_would
'll	_will
't	_not
've	_have
'm	_am

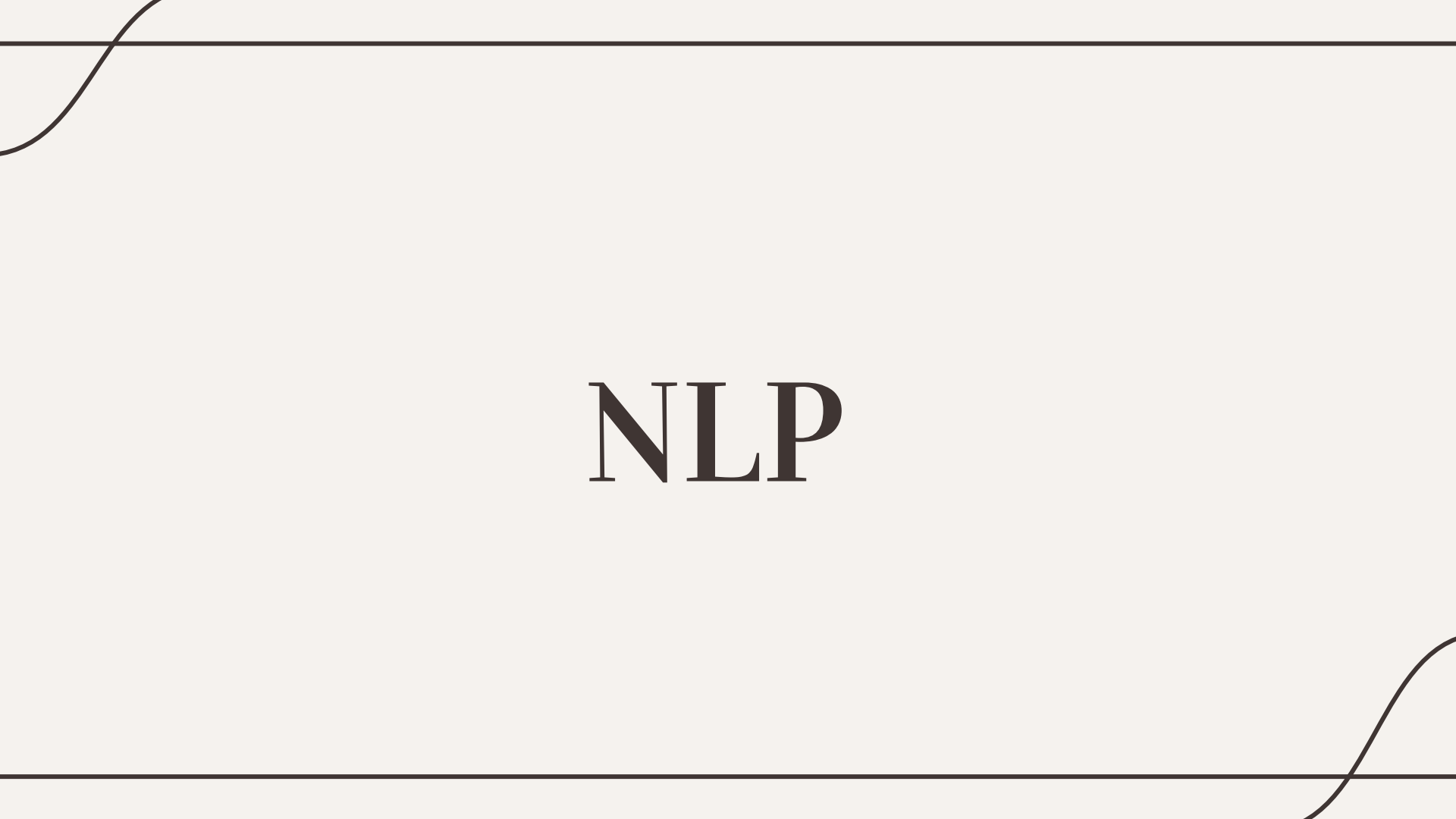
Example

```
import jiwer

sentences = ["she'll make sure you can't make it", "let's party!"]

print(jiwer.ExpandCommonEnglishContractions()(sentences))
# prints: ["she will make sure you can not make it", "let us party!"]
```

Source code in [jiwer/transforms.py](#)



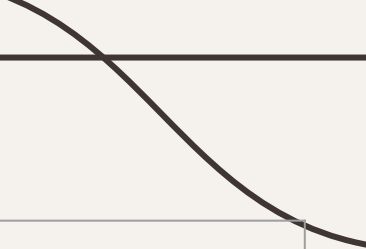
NLP

Processing

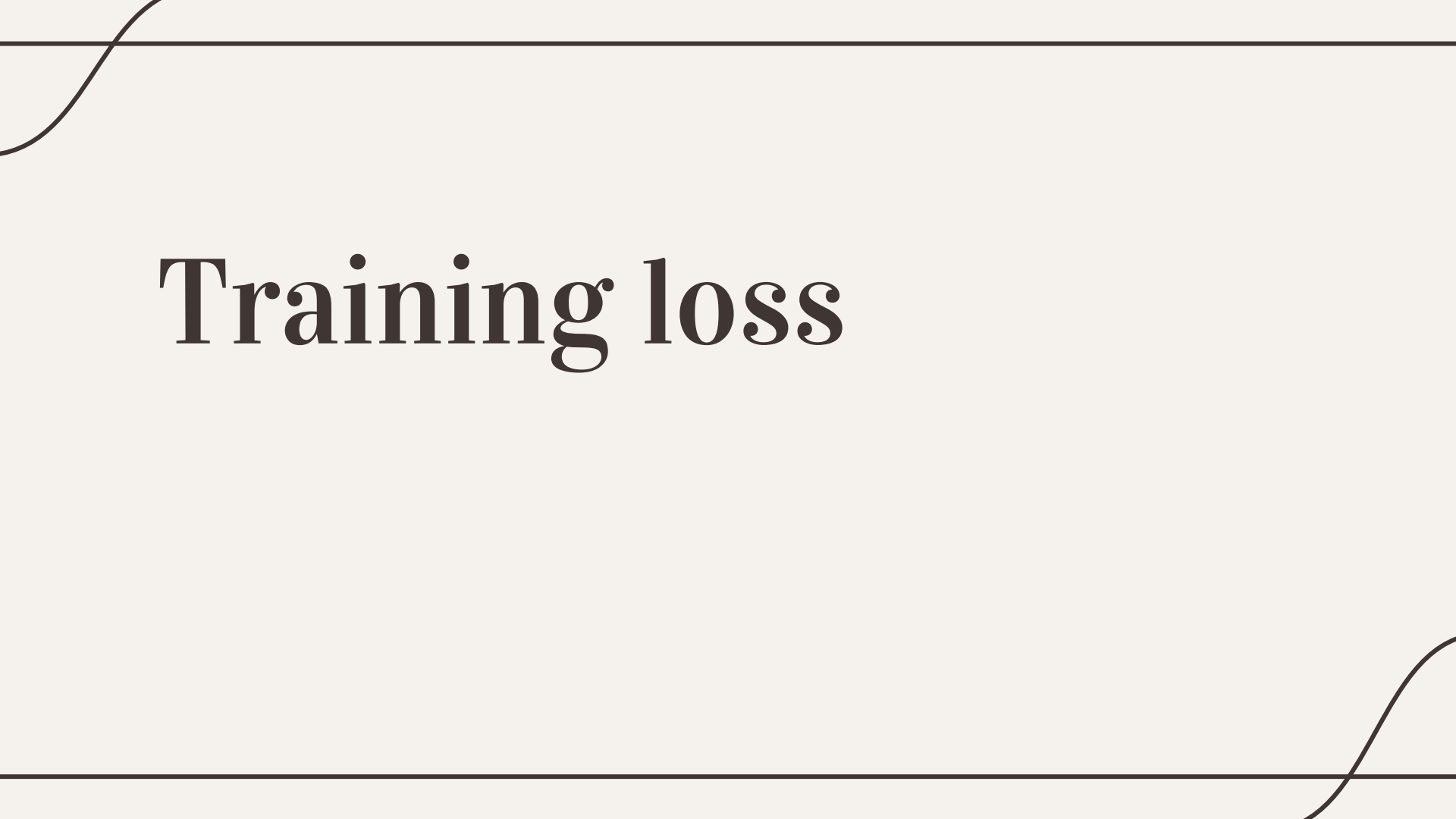
- **【state label】**: [BREATHING]、[GARBAGE]、[LIPSMACK]、[LAUGHTER]
 - 去除 only case
- 數字轉英文
- 狀聲詞同化

Data statistics

Number of data	9884
Number of data with state label	264
[BREATHING]	57
[GARBAGE]	44
[LIPSMACK]	1
[LAUGHTER]	163
Ratio	$9620 / 264 = 36.43...$



	turbo	large-v3	Parakeet	Canary
final	0.247	0.24054	0.26166	0.297
final denoise	0,252	0.30109	0.29661	0.3477



Training loss

	turbo	large-v3	Parakeet	Canary
final_target_set	0.247	0.24054	0.26166	0.297
direct denoise	0.298	0.30109	0.29661	0.3477
Wiener denoise	0.315	0.305	—	—
Noisereduce	0.3078	—	—	—
MetricGAN+ denoise	0.5262	—	—	—
Final_target_set (w/ jiwerPost Processing)	0.237437	0.227051		